

记录范围: 2026 年 4 月至 7 月 22 日。本文只使用工作区源码、.github/task-runs/、工程记忆和我在 AI.docx 中留下的同期感想。它不是学位论文，也不打算把个人经历包装成正式研究成果；这里只借用硕士研究生文档常见的版心、字号、行距和标题层级，把几个月里真正发生过的项目重新讲清楚。

我最想回答的并不是“AI 能不能写 RTL”，而是另一个更具体的问题：从 4 月第一次把一个 RV32I 核心交给 AI 协作，到 7 月让它围绕当前设计、任务合同、负向变体和证据闭环工作，中间究竟发生了什么？哪些目标真的完成了，哪些只是局部通过，哪些失败反而改变了后续路线？

task-run 的数量能够直观反映这段工程节奏：4 月 8 个，5 月 137 个，6 月 1246 个，7 月截至 22 日 549 个。不过，这些数字不能直接当成“完成任务数”。其中包含 focused run、e2e、复跑、负例、修订版和被保留的失败现场。对我来说，它们更像实验室记录：同一个判断要经过提出、执行、反证、回退和再发布，才会变成可继承的事实。

表 1: 四个月的项目主线与实际落点

月份	项目重心	当月真正推进的目标	月末仍然保留的边界
4 月	RV32I、NPC、NEMU	从 RTL 骨架推进到 AM 程序可运行、可追踪，并补齐同步 trap/CSR 主链	尚无完整 difftest、可恢复异常和真实 SoC 总线
5 月	流水线、Cache、SoC、OoO、RV64/Linux	从单核功能闭环推进到缓存、分支预测、ysyxSoC、RV64、Sv39 与真实 OpenSBI	CPI=0.5 未完成；真实 Linux 尚未完整启动
6 月	Ubuntu/NEMU 系统能力、AI 工程环境、OoO 架构重整	把系统软件长链、模块化 e2e、DB-first 召回、审查与证据包变成日常工程设施	大量局部门通过不等于整核签核；高风险投机方案被证伪
7 月	综合/STA/PPA、任务合同、变异验证、架构冻结	从“模型写代码”推进到 current-design 绑定、负向变体、fail-closed 检查和声明边界	full-core architecture freeze 仍为 GAP，PPA 仍为 UNQUALIFIED

还有一个必须先说清的事实：早期 task-run 主要记录“由 GitHub Copilot 还是 Codex 执行”，后期记录的是 Codex、agent-system、npc、rv64-linux 等角色，并没有稳定保存底层模型快照。因此本文不会把某个具体 GPT 或 Claude 版本硬贴到每一次运行上。AI.docx 中提到的 Claude Opus 4.6、DeepSeek v4 preview 和 GPT-5.5，是我对同期使用体验的回忆，不是 task-run 级别的可复现实验。当前整理环境可以

确认的是 Codex、GPT-5 系列；精确子版本同样不在本地证据里。这一处“不知道”，本身也是几个月后我最在意的工程问题之一。

1 2026 年 4 月：先让一个核心真正跑起来

1.1 项目从 RV32I 非流水线核开始

4 月 13 日的任务很直接：在 `npc/single/vsrc` 中落一版接近工程形态的 RV32I 非流水线核心。那时的工作不是从成熟 CPU 上做优化，而是面对几乎只有 `define.v` 和寄存器堆占位骨架的目录，把译码、立即数、ALU、比较、LSU、WBU 和顶层多周期状态机组织起来。

这次实现最后形成了 9 个 RTL 文件，并通过 Verilator lint。更重要的是，`task-run` 没把“lint 通过”写成“CPU 已经正确”：报告明确留下了三个缺口——没有仿真 `testbench`、没有镜像加载器、没有 NEMU 对拍；普通异常仍是 `halt-only`，尚未形成 `mtvec/mepc/mcause/mtval/MRET` 的可恢复闭环。现在回看，这种边界意识比首版代码本身更有价值。它让我很早就意识到，AI 最容易把“能编译”说成“已完成”，而工程记录必须强迫它把两者分开。

同一天的下一项工作，是在核心外面补 `NpcSimTop.sv`、DPI 和 C++ 仿真平台。最初目标只是把 AM 编译出的二进制装进 `pmem`，让程序能打印并正常退出；但项目很快从单文件 `main.cpp` 扩成参考 NEMU 的 `monitor -> cpu-exec -> paddr -> device/map` 分层。最终，`hello-riscv32-npc.bin` 在 NPC 上打印出 `Hello, AbstractMachine!`，以 `code=0` 退出；键盘链路也从主机标准输入穿过 NPC `keyboard`、KBD MMIO 和 AM `input.c`，在 `am-tests` 中观测到 `A DOWN/UP`。

这件事给我的第一条真实经验是：一个模型能写出模块，不等于它理解了工程；当镜像、地址空间、退出协议、设备 ABI、构建入口和观测接口被接成闭环以后，模型才第一次拥有“可纠错的环境”。没有这个环境，回答再像专家，也只是不可证伪的文本。

记录锚点：[.github/task-runs/2026-04-13-rv32i-nonpipe-core/task-report.md](#)；[.github/task-runs/2026-04-13-npc-dpic-bringup/task-report.md](#)。

1.2 从“能跑”到“能观察”

4 月 14 日的 `npc-trace-experience` 把 `itrace`、`mtrace` 和 `dtrace` 从编译期开关与零散日志点，改造成运行时可控的 `trace` 子系统。`batch` 模式能按条件开启 `trace`，`mon-`

itor 中可以查询和动态切换; mtrace 还专门排除了 ifetch, 只保留真实数据 load/store。这个看似不大的分类, 反映的是调试语义开始变得精确: 取指、数据访存和设备访问不再混成一条“有输出就算 trace”的日志。

紧接着, NEMU 的 `-b/--batch` 被补成完整接口, 又进入 Kconfig, 形成“默认关闭、命令行按次开启、Kconfig 持久开启”三条可验证路径。4 月 23 日, NEMU 的 RISC-V SYSTEM 核心补齐到 Zicsr、ECALL 和 MRET, AM CTE 的 yield 能连续输出 y。报告仍明确说明外部中断投递为空桩, 当前只覆盖同步 trap/CSR/MRET。这种“完成一个闭环, 同时声明下一个断点”的写法, 后来成为整个工作区的基本语气。

记录锚点: `.github/task-runs/2026-04-14-npc-trace-experience/task-report.md`; `.github/task-runs/2026-04-22-nemu-batch-mode/task-report.md`; `.github/task-runs/2026-04-22-nemu-kconfig-batch-mode/task-report.md`; `.github/task-runs/2026-04-23-nemu-system-core/task-report.md`。

1.3 当时的模型与我对它的判断

4 月上半月的 task-run 执行端主要写作 GitHub Copilot, 4 月下旬开始出现 Codex。底层具体模型没有被记录, 所以我只能诚实地说: 当时使用的是 Copilot/Codex 这两类工程入口, 而不是给每个任务补一个事后猜测的模型名。

从使用感受上看, 那时的模型仍然很像“放大版搜索和代码补全”。它能快速铺开一个 RV32I 骨架, 也能沿着已有 Makefile 和源码继续工作, 但一旦缺少测试平台, 它很难自己判断哪些只是语法成立, 哪些已经形成系统行为。4 月 21 日补齐根 AGENTS.md、CLAUDE.md、GEMINI.md、Cursor/Windsurf 等兼容入口, 表面上是在做工具兼容, 实质上是在提出一个更前瞻的问题: 如果未来模型会不断变化, 工程知识就不该锁死在某一个产品的提示词里, 而应该以模型无关的仓库契约存在。

与现在的 Codex/GPT-5 系列相比, 4 月的差异并不只是“模型更聪明了”。真正的差异是现在的模型进入工程时, 已经有入口规则、记忆、task-run、profile 和证据边界; 4 月的模型主要面对文件, 7 月的模型面对的是一个可以召回、执行、反证和留痕的系统。

2 2026 年 5 月: 从功能核走向微架构与 Linux

2.1 流水线、Cache 与分支预测开始形成数据闭环

5 月的任务密度突然上升。5 月 19 日以后, NPC 连续经历流水线拆分、控制/数据通路解耦、DPI 事件回调、单模块 testbench、I/D Cache、AXI、CSR 与中断等

改造。这里最典型的是 5 月 20 日的 cache SRAM/AXI 任务：ICache 和 DCache 的 tag/data 被改成 SRAM-like wrapper，DCache 采用 write-back/write-allocate，miss、dirty victim、uncached access 走分离的 AXI 路径，fence.i 必须先写回脏数据再失效 I/D Cache。

这次任务不是只看波形说“似乎能工作”。记录里留下了 21/21 cache test PASS、流水线 test PASS、Verilator build PASS，以及 AM load-store、fence-i 回归；同时又保留了限制：当时仍是单 beat、32-bit、无 ID、无 burst、无多 outstanding 的 AXI4-Lite 风格子集，并且没有综合/STA。这个边界很关键，因为“接口叫 AXI”与“已经达到真实 AXI4 SoC 能力”完全不是一回事。

5 月 21 日的 BPU local-history 任务则第一次把优化与数字直接联系起来。根据 task-run，branch miss 从用户给出的 3 429 214 基线降到约 2 457 500；随后把方向表初值改成弱跳转后为 2 458 726，CPI 保持 1.285，说明该追加改动基本中性略负。这段记录让我学会了一件很重要的事：优化不是“代码更复杂就更先进”。一个改动如果不能在固定口径下解释统计变化，就只是一种结构偏好；如果结果略负，也应该原样留下，而不是用叙事把它包装成收益。

记录锚点：.github/task-runs/2026-05-20-npc-cache-sram-axi/task-report.md；.github/task-runs/2026-05-21-npc-bpu-local-history/task-report.md。

2.2 接入 ysyxSoC，而不是停在自建仿真壳

5 月 23 日，NPC 增加了符合 ysyxSoC 约定的 CPU 顶层，ysyxSoCFull 可以实例化 ysyx_26010035。当时的验证包括 NPC lint、SoC lint、构建和 1000 周期最小复位仿真，也包括直接实例化核心的 Icarus testbench。报告特意提醒：这个 smoke 只证明接线、构建和最小运行入口成立，不等于完整 SoC 软件栈已经通过。

5 月 24 日的 B2 cache closure 又把平台差异真正暴露出来。MROM/uncached 取指必须保留 RVC 半字 PC 的 byte offset；旧 mem-test 只能证明 SRAM byte lane 与符号扩展，不能证明 DCache write-back 和 dirty refill。修复后，single 后端保持 26/26 PASS，ysyxSoC 的 mem-test 与 char-test 通过；随后 I/D Cache 继续升级为 2-way set-associative，并分别在 single 和 SoC 上复跑。这里体现出的不是“AI 一次写对”，而是它能沿着地址图、RVC、缓存行、平台后端和回归集合继续追根因。

记录锚点：.github/task-runs/2026-05-23-npc-ysyx-soc-integration/task-report.md；.github/task-runs/2026-05-24-b2-ysyxsocfull-cache/task-report.md。

2.3 OoO 与 CPI=0.5: 目标可以很远, 但检查点必须真实

5 月 28 日, 我把长期目标写得非常激进: 完成一个乱序超标量处理器, CPI=0.5。第一阶段先落 issue queue、ROB、rename map 等基础件, 并保持主路径回归; 当时 3 个新增 testbench、全量模块 30/30、NPC lint/build 和 cpu-tests add 均通过, 但 OoO 模块尚未完全接管主路径, 因此报告没有把它写成“OoO 核已完成”。

5 月 29 日的一个检查点记录了 add 的 541 cycles、839 commits、CPI=0.645。更有价值的是, 它同时记录了一次 issue queue 正确性 bug, 以及一个修复后性能退化到 CPI=0.665、最终撤回实验的过程。CPI=0.5 在 5 月没有完成; 这不是文章需要遮掩的失败, 反而说明目标开始从口号变成可以被数字拒绝的工程命题。

我后来越来越相信, 前瞻力不是把目标说得保守, 而是敢于提出远目标, 同时建立不会替它“圆场”的验证制度。只有失败能够被稳定记录, 目标才是真的, 而不是宣传语。

记录锚点: [.github/task-runs/2026-05-28-ooo-superscalar-bootstrap/task-report.md](#); [.github/task-runs/2026-05-29-ooo-cpi-645-iq-fix/task-report.md](#)。

2.4 从 RV32 迁到 RV64, 再走向真实 OpenSBI 与 Linux

5 月 29 日, npc/rv64 后端被建立起来, ARCH=riscv64-npc -> npc/sim BACKEND=rv64 -> NpcSimTop -> RV64 core 的链路打通, 基础 CPU-test 38/38 PASS。报告同时说明, 当时的 NEMU RV64 reference 不完整, 不能据此声称 RV64 difftest 已经成立; bitmanip 和 compressed 也不在这一轮通过范围内。

5 月 30 日以后, 项目很快越过“64 位算术”进入 Linux 的真实前置条件: S-mode CSR、delegation、SRET、A 扩展原子访存、Sv39 page walk、PTE 权限、SBI、PLIC、UART 和多镜像加载。rv64-linux-prereq 在保持 40/40 CPU-test 的同时, CoreMark 记录为 CPI=0.779; rv64-sv39-bridge 实现 1GB/2MB/4KB leaf 的成功路径与权限检查, 但明确不宣称完整 Linux boot。

随后, 真实 OpenSBI v1.8 被跑起来。一个很生动的根因是: OpenSBI 固件布局不满足 fw_rw_offset 的 power-of-2 约束, 修正后又遇到 semihosting magic sequence 中的 ebreak 被 NPC 当成 AM halt。把这类具体边界处理清楚后, OpenSBI banner 完整输出, mini S-mode payload 打印 S 并 GOOD TRAP。到 5 月 31 日, 真实 Debian RISC-

V kernel 已经被加载，项目越过 mimpid 缺口，并定位到 Linux relocate_enable_mmu 中 RAS 低地址返回项与高半区地址切换的冲突。

但月末结论仍然是：真实 Linux kernel 尚未完整 boot，只能证明已经越过 OpenSBI mimpid 与 early MMU relocation/RAS 缺口，并在观测窗口内持续退休。console、rootfs、virtio-blk、多源 PLIC 和后续 kernel 路径仍待收敛。把“已经走到哪里”精确到启动阶段，比笼统写一句“支持 Linux”更有技术含量。

记录锚点：[.github/task-runs/2026-05-29-npc-rv64-backend/task-report.md](#); [.github/task-runs/2026-05-30-rv64-linux-prereq/task-report.md](#); [.github/task-runs/2026-05-30-rv64-sv39-bridge/task-report.md](#); [.github/task-runs/2026-05-30-rv64-opensbi-mini-boot/task-report.md](#); [.github/task-runs/2026-05-31-rv64-real-linux-mimpid-ras/task-report.md](#)。

2.5 当时的模型：能力跃迁真实存在，但不能只归因于版本号

5 月的 task-run 执行端以 Codex 为主。我的同期笔记还记录了几次并行体验：Claude Opus 4.6 能在 agent 模式下理解多文件、Makefile 和 NPC 环境；一次 DeepSeek v4 preview API 尝试在流水线 RTL 上花费约 70 元却没有完成；同类问题后来由我笔记中称作 GPT-5.5 的模型在约半小时内解决。这个经历促使我开始持续使用当时的 SOTA 模型。

不过，这些都是个人现场感受，不是固定提示词、固定代码版本、固定测试集的模型评测。我能得出的可靠结论只有两个。第一，模型代际差异确实已经大到足以改变复杂 RTL 任务的可行性；第二，同一个强模型如果没有工作区结构、编译入口、测试与记录，也会迅速退回“生成一段看似合理代码”的状态。5 月 28 日打包出的 AI 硬件开发环境包含 66 个文件、约 916K，并经过凭证与路径脱敏扫描。那一刻我已经不再把提示词当核心资产，而是开始把环境本身当作可迁移的工程能力。

3 2026 年 6 月：模型开始成为系统工程的一部分

3.1 Ubuntu 的判据从“有输出”变成分层 gate

6 月 1 日的 rv64-ubuntu-shell-gate 很能说明方法的变化。QEMU 已经证明 Ubuntu Base 的 /bin/sh-c 能执行，NPC 也能生成对应 OpenSBI/DTB/initramfs 产物，但报告明确写道：不能把 QEMU full shell PASS 或 NPC syscall-only probe PASS 升级成 NPC 官方 Ubuntu /bin/sh PASS。也就是说，同一个“shell”被拆成了镜像正确性、参考平台、NPC ISA/FP 能力、系统调用与真实执行路径几个不同层级。

随后，RV64 浮点加减乘除、开方、转换、比较、FCSR，TLB/paged cache、UART/PLIC、rootfs/virtio 等任务连续推进。NEMU 一侧则从能启动内核，扩展到 systemd、virtio-blk、virtio-net、virtio-rng、UART getty、GDB stub、QMP、SSH、apt/dpkg、DNS、NTP、网络转发、systemd resource control、用户管理和 simplefb/fbcon 等大量真实软件链路。这里不应把每个 task-run 都写成一项“功能已永久完成”；更准确的说法是，这些能力各自进入 contract/focused/full-gate，并在对应配置与观测窗口内留下了可复跑证据。

6 月 6 日，nemu-ubuntu profile 已能生成由 recall、tool environment、NPC/RV64 contract、NEMU static 与 slice contract 组成的模块化证据包。到 6 月 22 日和 25 日，nemu-dev-full-gate 被用于 OpenSSH 文件传输、systemd-analyze 等具体系统行为。工程对象已经不再只是指令解释器，而是一台需要同时处理设备、用户态、服务管理和长时间运行判据的系统机器。

记录锚点：[.github/task-runs/2026-06-01-rv64-ubuntu-shell-gate/report.md](#)；[.github/task-runs/2026-06-06-nemu-ubuntu-production-e2e/task-report.md](#)；[.github/task-runs/2026-06-22-2026-06-22-nemu-full-openssh-file-transfer-full-gate/task-report.md](#)；[.github/task-runs/2026-06-25-nemu-full-systemd-analyze-boot-full-gate/task-report.md](#)。

3.2 性能优化开始接受大量负结论

6 月 15 日至 16 日集中出现了 NEMU Ubuntu 的 opcode mix、decode cache、wide ifetch、RVC fast path、PMP cache、TB 大小、host perf 等大量 profile。很多名字后面带着 revert、negative、rerun。如果只看最终源码，这些尝试几乎像没有发生；task-run 却把它们保留下来，使“为什么没有采用某个看起来很快的优化”变得可解释。

这段经历让我对 AI 优化产生了更成熟的判断。模型很擅长快速生成候选，也因此更容易制造局部加速、全局退化或测量口径漂移。真正稀缺的不是候选数量，而是固定输入、A/B、性能计数、回归、回退和残留检查。负结论并不是浪费，它们构成搜索空间的边界；没有这些边界，模型会在几轮对话以后再次提出同一个坏主意。

3.3 AI 环境从文档集合升级为可执行治理系统

6 月 11 日是另一个分水岭。scripts/github_index_db.py 开始把 .github 下的 memory、task-run 和证据做 SQLite 索引，支持 build、query、doctor 和按 profile 召回。原始 Markdown 仍然是事实源，数据库只负责有界检索；这避免了让模型每次手工遍历成千上万个文件，也避免了数据库反过来“拥有”原始事实。

与此同时，模块化 e2e、profile catalog、context brief、run manifest、evidence index、runtime artifact boundary、state traceback、Reviewer/Inspector 路由和 commercial delivery readiness 逐渐形成。AI 不再只收到一句“修好这个 bug”，而是先恢复状态、分类任务层级、选择图、执行节点、验证、审查，再把稳定结论写回。到这一步，我开始把 AI 环境看成一种工程操作系统：模型是可替换的计算单元，规则、记忆、工具和证据协议才决定它能否长期工作。

记录锚点：[.github/task-runs/2026-06-11-github-index-db-first-migration/task-report.md](#);
[.github/task-runs/2026-06-13-commercial-delivery-final/task-report.md](#)。

3.4 回到 RV64：先写“架构宪法”，再接受高风险方案被证伪

6 月下旬，工作重心又回到 RV64 OoO 核。6 月 27 日的 core control-flow decomposition 把 direct control-flow、predictor update、CSR illegal probe、synthetic lane1 retire 等职责从 core glue 下沉到对应模块；focused 5/5、默认模块 103/103、lint 和 build 均通过。这个阶段的目标不是增加新指令，而是把未来优化所依赖的责任边界重新整理清楚。

6 月 29 日形成的“微架构宪法”更像一次系统审计：记录称，审计过程消耗约 849k subagent tokens、360 次工具调用，最终把前端、执行域、恢复路径和架构缺口固化成可讨论的事实。随后，一个高风险方案被真正点火：尝试复活休眠的分支投机。结果是 riscv-tests 255/16、AM 25/32，多项存储、长操作和分支程序失败或活锁。回退以后恢复到 riscv-tests 271/0、模块 113/113 和 lint/style 全绿。

这次失败非常重要。它证明“把开关打开”不是通往真 OoO 的捷径，也把后续路线从模糊的“继续修投机”改成 ROB-walk、rename restore、IQ squash 和 branch+jump 一体化恢复。6 月 30 日的 mode=1 bring-up 仍明确标为未完成，但已经能运行数百条指令，并把剩余根因继续收窄。高水平工程并不意味着每个大胆方案都成功，而是失败以后能留下足够的信息，使下一轮不必从猜测重新开始。

记录锚点：[.github/task-runs/2026-06-27-npc-rv64-ooo-core-controlflow-decompose/task-report.md](#);
[.github/task-runs/2026-06-29-rv64-ooo-core-architecture-constitution/report.md](#);
[.github/task-runs/2026-06-30-rv64-ooo-b2-rob-walk-mode1-bringup/report.md](#)。

3.5 当时的模型：从单一助手变成有角色的协作网络

6 月的 task-run 已经很少只写“Codex 完成某文件”，而是出现 rv64-linux、rv64gc-userland、nemu、software-flow、hardware-flow、agent-system、Reviewer 和 Inspector 等角色。底层模型版本仍没有可靠记录，但协作形态发生了实质变化：同一个模型不再同时扮演需求解释者、实现者和验收者；不同节点拥有不同输入、输出和证据责任。

和当前 Codex/GPT-5 系列相比，6 月最重要的过渡不是推理分数，而是我开始主动限制模型。它必须承认 QEMU PASS 不能替代 NPC PASS，profile PASS 不能替代 Linux/full-core/PPA，静态合同不能替代动态行为。这种限制并没有削弱 AI，反而让它第一次能稳定参与长期项目。模型越强，越需要一个能拒绝它的环境。

4 2026 年 7 月：从“做出结果”走向“证明结果属于当前设计”

4.1 Yosys、STA 与 200 MHz：工具跑通以后，才开始讨论证据资格

7 月初，项目把综合和时序正式接进来。最初的 Yosys/NPC profile 甚至留下 blocked 记录：npc-rv64-contract 失败，所以不能把后续判断建立在那个节点上。我的同期笔记还记录过一次很典型的环境问题——综合结果落在 /tmp，模型跑完命令却没有把产物变成可继承证据。后来 versioned Yosys/STA flow、输入冻结、source-to-netlist 绑定和 evidence index 才逐渐补齐。

7 月 11 日的 rv64-200mhz-program 状态是 in_progress_f0_complete，并明确写着 goal_complete=false：功能完整与最终物理 200 MHz 必须同时闭合，pre-layout 5 ns 只是阶段 gate。7 月 15 日的记录更能说明边界。有的 fresh exact-5ns proxy 点得到最差 slack +0.179479554 ns 或 +0.131591633 ns，TNS、violations、loops 为 0；另一个候选为 +0.059167784 ns，虽然 200 MHz proxy 非负，却没有达到额外 +0.10 ns promotion reserve，而且九项架构门仍为 RED，功耗也未资格化。因此正确结论不是“芯片已在 200 MHz 签核”，而是“部分约束下存在通过 5 ns proxy 的候选，完整物理与 PPA 结论仍不成立”。

这体现了我在几个月内形成的判断：频率不是一个命令输出，而是一组绑定关系。RTL、配置、网表、库、约束、corner、路径例外、功能回归和报告解析必须属于同一个设计快照；其中任何一项没有闭合，MET 都可能只是一个漂亮但无资格的词。

记录锚点：[.github/task-runs/2026-07-07-yosys-rv64-synth-probe-npc-dev/task-report.md](https://github.com/task-runs/2026-07-07-yosys-rv64-synth-probe-npc-dev/task-report.md)；[.github/task-runs/2026-07-11-rv64-200mhz-program/task-report.md](https://github.com/task-runs/2026-07-11-rv64-200mhz-program/task-report.md)；[.github/task-runs/2026-07-15-rv64-ppa-architecture-recovery/task-report.md](https://github.com/task-runs/2026-07-15-rv64-ppa-architecture-recovery/task-report.md)。

4.2 task-run 不再只是日志，而成为任务合同的执行面

7 月 17 日以后，AI workflow 继续强化：Database/Skill/Agent 三层、运行态产物边界、状态回退、Reviewer/Inspector、bounded brief、strict guard、RTL 子 agent 合同和 shell ownership 被写进可执行 gate。中间也保留了 blocked 的 run，例如 7 月 18 日的 agent-system closure 与 7 月 19 日的 RTL generation workflow。它们没有被删除，而是作为“当时输入下不能发布结论”的反例存在。

到了 7 月 20 日的 v8q 双内存共享 AXI miss fabric F0，任务报告已经具备完全不同的形态：需求、事件、状态、优先级和响应归属先写进合同；release、OOO_ASSERT 和 assert-negative profile 分开运行；12/12 个能够编译、例化和激活的源码变体都必须被目标检查拒绝；runner 还要证明 source 在运行前后没有变化。这个 leaf 被允许标为局部 GREEN，但整体仍保持 RED，PPA 为 UNQUALIFIED，因为 F0 尚未实例化，也没有证明两路真实 cache hit、独立翻译、MIQ/completion 或物理 SQ byte query。

这正是我认为最值得强调的前瞻性：不是让 AI 给自己打一个 PASS，而是让它先写出“什么错误必须被发现”，再主动制造这些错误。验证从正向样例变成对 checker 灵敏度的测试，模型从代码生成器变成了证据生产者，同时仍受独立审查者约束。

记录锚点：[.github/task-runs/2026-07-17-rv64-ppa-ai-workflow-final-2/task-report.md](#)；[.github/task-runs/2026-07-20-rv64-v8q-dual-memory-datapath/task-report.md](#)。

4.3 “架构冻结检查通过”与“架构已经冻结”是两回事

7 月 21 日的 v9b full-core arch-stable freeze 是一份很成熟的负责任记录。canonical runner 的 38/38 workflow 测试、current audit 和 current verify 都返回 0；九项 DI-1 至 DI-5、OOO-1 至 OOO-4 定向门在同一设计上为 9/9 GREEN。但 full-core 汇总仍是 113 PASS / 46 GAP，architecture_freeze=GAP，ppa=UNQUALIFIED。也就是说，冻结资格检查器通过了自己的合同，却没有替未完成的架构条件签字。

这种表达看似保守，实际上比一句“全绿”更能体现水平。它区分了三件事： workflow 本身是否可信，当前已有定向能力是否通过，以及系统级发布条件是否齐全。只要 46 个 GAP 仍在，前两者就不能被用来偷换第三者。

记录锚点：[.github/task-runs/2026-07-21-rv64-v9b-arch-stable-freeze/task-report.md](#)。

4.4 当前检查点：IFU-ACCESS-G1 与 19 个负向 RTL 变体

7 月 22 日的 v9i 是目前最能代表这套方法的例子。任务不再泛泛地说“检查 IFU”，而是把当前设计重新绑定到 IFU-ACCESS-G1：沿 `OooFetchAxiBridge.v`、`PmpChecker.v`、`AxiXbar.v`、`OooFetchPacketDecode.v` 和 `OooFetchHeadPairGate.v` 追踪 16/32 位取指 footprint、RVALID/RREADY、2B beat、8B PTW、lane1 offset/visibility 与 fault ownership。

当前记录给出的局部证据是：2B 读取 oracle PASS，编译成功的负向 RTL 变体 19/19 被拒绝，证据单测 10/10；审查者在冻结矩阵和这些变体中没有发现共同盲点。与此同时，报告明确限制结论：它只覆盖当前冻结材料和 IFU-ACCESS-G1，不替代 IFU-TVAL、PTW-PMP、完整取指子系统、full-core、Linux 或 PPA。更上层的 full-core audit 依然是 `architecture_freeze=GAP`、`ppa=UNQUALIFIED`。

如果把 4 月和 7 月放在一起看，变化非常明显。4 月的问题是“这段 RTL 能不能 lint”；7 月的问题变成“这份结论是否绑定当前 design-id，checker 是否能拒绝 19 类会编译的真实错误，runner 是否保持 source unchanged，局部门是否被错误外推到系统层”。模型能力当然提高了，但更重要的是，我已经把“相信模型”改造成“设计一套让模型必须提供反证机会的流程”。

记录锚点： `.github/task-runs/2026-07-22-rv64-v9i-ifu-access-current-design/task-report.md`;
`.github/task-runs/2026-07-22-rv64-ifu-access-current-v9i-3/task-report.md`。

4.5 当时与现在的模型对照

表 2 模型身份、工程形态与可确认边界

时间	记录能确认的执行端	模型在项目中的实际形态	与当前环境的差异
4 月	GitHub Copilot, 后转 Codex; 底层版本未记录	读取源码、生成模块、补接口, 依赖人工指出验证缺口	当前 Codex/GPT-5 系列进入前已有规则、记忆与 task-run
5 月	Codex; 同期个人笔记另提 Opus 4.6、DeepSeek v4 preview、GPT-5.5	能处理多文件流水线、Cache、SoC 与 Linux 长链, 但模型比较不是受控实验	当前更强调任务合同与结论边界, 不以一次成功代表模型普遍能力
6 月	Codex 与 rv64-linux、nemu、agent-system 等角色; 底层版本未记录	从单助手转为分工图, 能够召回、执行 profile、生成证据包并接受审查	当前增加 source/design 绑定、revision、strict guard 与 fail-closed 发布
7 月	Codex、多角色 Reviewer/Inspector、受约束 RTL 子 agent; 底层版本未记录	围绕 current design、mutation、negative profile、architecture gate 工作	当前最显著的能力不是“写得更多”, 而是能证明局部结论没有越界

我的结论是：模型版本重要，但模型版本不是项目能力的唯一坐标。4 月到 7 月，执行端从 Copilot/Codex 演化为带角色的 Codex workflow；同期笔记中的 Opus 4.6 和 GPT-5.5 体现了模型代际进步；真正让这些能力稳定下来的，却是工作区本身。今天即使换一个更强模型，如果它绕过 task-run、忽略 design binding、看不到失败证据，工程水平仍可能倒退到 4 月。

5 回看这四个月：我真正构建的是什么

最初，我以为目标是得到一个“会写 RTL 的 AI”。现在我更愿意说，我在构建一套让模型能够参与芯片工程的方法：它要在边界明确、输入有界、状态可恢复、证据可追溯、结论可反驳的条件下工作。模型仍会错，testbench 也可能错，PASS 甚至可能是假绿；因此系统必须允许审查者主动寻找反例，并允许一个很努力的任务最终以 RED、GAP 或 UNQUALIFIED 收尾。

这四个月里，项目目标也发生了四次变化：

1. 4 月的目标是把 RV32I 核心、DPI、AM 与 NEMU 接成最小可运行闭环；

2. 5 月的目标扩展为流水线、Cache、分支预测、ysyxSoC、OoO、RV64 与 Linux 前置能力；
3. 6 月开始同时建设完整系统软件路径与 AI 工程基础设施，并用失败实验约束微架构路线；
4. 7 月则把重点推到综合/STA/PPA 资格、current-design 绑定、变异验证和 full-core 发布边界。

这些工作还不能证明“AI 已经独立完成商业处理器”，也不能证明完整 Linux、物理 200 MHz 或 PPA 已经签核。相反，工作区里最有价值的证据之一，恰恰是它明确保存了 `goal_complete=false`、`architecture_freeze=GAP` 和 `ppa=UNQUALIFIED`。我认为真正的高水平不在于回避这些词，而在于知道为什么此刻只能写这些词，以及下一步需要什么证据才能把它们改变。

接下来，我最希望继续推进四件事。第一，把模型供应商、产品入口、基础模型快照、reasoning 配置、工具版本和提示词修订号写入 run manifest，使“当时是什么模型”不再依赖回忆。第二，把上千个 task-run 组织成有父子关系的实验图，区分一次任务的候选、复跑、否决与最终发布，避免数量掩盖主线。第三，继续把 leaf、aggregate、full-core、Linux 和 PPA 分成不可越级的证据层。第四，探索专用 XPU 或 ASIC 对 agent 工作负载的加速：未来模型能力会继续增长，但真正昂贵的将是长上下文召回、并行验证、变异生成和全链证据处理，而不是单次代码补全。

我不需要等一个定义模糊的 AGI 再开始准备。只要模型已经能在确定性流程里超过我完成某些局部工作，就值得提前建立责任边界、证据协议和可复现环境。前瞻性不表现为对 AI 作最大胆的承诺，而表现为在它还不稳定的时候，就把未来规模化协作所需的基础设施先做出来。

本文的所有工程数字和状态均按对应 task-run 的局部口径复述。它们用于记录个人思考，不替代源码审查、完整回归、Linux/full-core 验收、形式验证或物理实现签核。